CliniScan:Lung-Abnormality Detection on Chest X-rays using AI

Milestone - I

DatasetUsed:
VinDr-CXR
PlatformUsed:
Kaggle
Submitted By

Afreen Nisha F
BTech CSE(AI & ML) - 2nd year
Lovely Professional University,Punjab
Date:02.03.2026
Mentor:
Miss.Meghana Andhavarappu

TABLE OF CONTENTS

SL.NO.	Section	PageNo. (Est.)
1	Title page	i
2	Table of Contents	ii
3	Milestone 1 – Objective and Tasks	iii
4	Task No. 1	iv
5	Task No. 2	v-vii
6	Task No. 3	viii-xi
7	Task No. 4	xii

Milestone 1

Data Preparation & Initial Setup (Weeks 1-2)

● Objective:

Acquire and prepare the VinDr-CXR dataset, set up the development environment, and create initial data processing pipelines.

● Tasks:

1. Download the VinDr-CXR dataset.
2. Develop scripts for converting DICOM images to PNG/JPEG.
3. Develop scripts for parsing and converting annotations (CSV to YOLO/COCO format).
4. Set up the development environment with PyTorch/TensorFlow, pydicom, pandas, OpenCV.

TASK NO. 1

Download the VinDr-CXR dataset

The dataset used for this project is the VinDr-CXR dataset, which contains a large collection of chest X-ray images in DICOM format along with structured annotation data in CSV format. The dataset is publicly available on the Kaggle.

Due to the large size of the dataset (several gigabytes), downloading it to a local system was not practical. Instead, the dataset was accessed directly within the Kaggle cloud notebook environment. Kaggle provides an integrated feature that allows users to attach datasets directly to notebooks without manual downloading.

The following steps were performed:
Opened a new Kaggle notebook.
Used the "Add Data" option to attach the VinDr-CXR dataset.
Verified that the dataset appeared under the /kaggle/input/ directory.
Confirmed the presence of DICOM image folders and the annotation CSV file.

To validate successful access, the annotation CSV file was loaded using the pandas library, and sample DICOM images were accessed from the image directory. The successful reading of files confirmed that the dataset was properly integrated and ready for preprocessing.

Although the dataset was not downloaded to a local machine, it was successfully acquired and made fully accessible within the Kaggle cloud environment, fulfilling the objective of Task 1.

TASK NO.2

Develop scripts for converting DICOM images to PNG/JPEG

The VinDr-CXR dataset contains chest X-ray images in DICOM format. Since deep learning models and computer vision frameworks work more efficiently with standard image formats, a preprocessing script was developed to convert DICOM images into JPG format.

Libraries Used

The following Python libraries were used in the conversion pipeline:
pydicom – to read DICOM files
numpy – for numerical operations
opencv (cv2) – for resizing and saving images
tqdm – for progress tracking
os – for file path management

Conversion Process

A loop was created to iterate through all image IDs. For each image, the following steps were performed:

1. Read the DICOM File

The DICOM file was loaded using:
pydicom.dcmread(dicom_path)

The pixel data was extracted using:
dicom.pixel_array

2. Convert to Float Format

The pixel array was converted to float32 to allow normalization:
image = dicom.pixel_array.astype(np.float32)

3. Normalize Pixel Values (0–255)

Since DICOM images may contain a wide range of intensity values, normalization was applied:
image = (image - image.min()) / (image.max() - image.min()) image = (image * 255).astype(np.uint8)

This ensures:

Pixel values are scaled between 0 and 255
Image format becomes compatible with JPG saving

4. Resize Images

All images were resized to a fixed resolution of **1024 × 1024 pixels** using OpenCV:
image = cv2.resize(image, (1024, 1024))
Resizing ensures uniform input dimensions for future model training.

5. Save as JPG Format

The processed image was saved in JPG format with 95% quality:
cv2.imwrite(save_path, image, [cv2.IMWRITE_JPEG_QUALITY, 95])
This maintains high visual quality while reducing file size.

6. Error Handling and Monitoring

A try-except block was used to handle corrupted or unreadable files.
tqdm was used to display a progress bar during batch processing.
A completion message confirmed successful execution.

Verification

The conversion process was verified by:

Checking the output directory for generated JPG images.
Displaying sample converted images.
Confirming uniform image size (1024×1024).
Observing successful completion without major errors.

Conclusion

A complete DICOM-to-JPG conversion pipeline was successfully implemented. The images were normalized, resized to 1024×1024 resolution, and saved in high-quality JPG format. This preprocessing step ensures that the dataset is standardized and ready for annotation processing and deep learning model development in subsequent milestones.

CODE:

```
import pydicom
import cv2
import numpy as np
from tqdm import tqdm

image_ids = [f.replace(".dicom", "") for f in os.listdir(TRAIN_DICOM_PATH)]

for image_id in tqdm(image_ids):

    dicom_path = os.path.join(TRAIN_DICOM_PATH, image_id + ".dicom")
    save_path = os.path.join(IMAGE_SAVE_PATH, image_id + ".jpg")

    try:
        dicom = pydicom.dcmread(dicom_path)
        image = dicom.pixel_array.astype(np.float32)

        # Normalize to 0-255
        image = (image - image.min()) / (image.max() - image.min())
        image = (image * 255).astype(np.uint8)

        # Resize to 1024×1024
        image = cv2.resize(image, (1024, 1024))

        # Save as JPG
        cv2.imwrite(save_path, image, [cv2.IMWRITE_JPEG_QUALITY, 95])

    except Exception as e:
        print("Error:", image_id)

print("DICOM → JPG conversion completed.")
```

TASK NO. 3

Develop scripts for parsing and converting annotations (CSV to YOLO/COCO format)

Parsing and Verification of Annotation CSV File

The VinDr-CXR dataset provides annotation information in CSV format, containing image IDs, abnormality labels, and bounding box coordinates. In order to prepare the data for object detection model development, the annotation file was first loaded and verified.

Loading the CSV File

The annotation file was loaded using the pandas library:
The read_csv() function was used to read the CSV file.
The dataset was stored in a DataFrame for further processing.
The total number of rows was printed to verify successful loading.
After loading the file, the output confirmed:
Total Rows: 67,914
This indicates that the annotation file was successfully read without any errors.

Initial Data Inspection

To understand the structure of the dataset, the first few rows were displayed using:
df.head()
This step allowed verification of:
Column names
Data types
Presence of image identifiers
Bounding box coordinate fields
Class labels
Displaying the first few rows ensured that the dataset structure matched the expected format required for further processing.

Validation Outcome

The successful execution of this step confirms that:
The annotation CSV file is accessible.
The file is not corrupted.
The dataset contains 67,914 annotation entries.
The data is ready for transformation into YOLO/COCO format in the next stage.

CODE:

```
import pandas as pd

df = pd.read_csv(CSV_PATH)

print("CSV Loaded. Total Rows:", len(df))
df.head()
```

```
CSV Loaded. Total Rows: 67914
```

Conversion of CSV Annotations to YOLO Format

After successfully loading and verifying the annotation CSV file, the next step involved converting the bounding box annotations into YOLO format. Object detection models such as YOLO require annotation files in a specific normalized format rather than raw pixel coordinates.

Grouping Annotations by Image ID

The dataset contains multiple bounding boxes per image. Therefore, the DataFrame was grouped using:
grouped = df_filtered.groupby("image_id")
This ensured that all annotations corresponding to a single image were processed together and saved into one label file.

YOLO Annotation Format

YOLO requires each bounding box to be written in the following format:
class_id x_center y_center width height
Where:
x_center and y_center are normalized center coordinates
width and height are normalized bounding box dimensions
All values are scaled between 0 and 1

Conversion Process

For each image:

1. A .txt label file was created using the image ID.
2. Image dimensions were fixed at **1024 × 1024**.
3. For each bounding box:
   Extracted x_min, y_min, x_max, y_max

Calculated center coordinates:

x_center=((x_min+x_max)/2)/img_w
y_center = ((y_min + y_max) / 2) / img_h

Calculated normalized height:

height = (y_max - y_min) / img_h

4. Wrote the converted values into the corresponding .txt file.
   A loop with tqdm was used to monitor progress during batch processing.

Output Generation

One .txt file was generated per image.
Each file contains all bounding box annotations for that image.
Label files were stored in the designated label directory.
The process concluded with the confirmation message:
CSV → YOLO conversion completed.

Verification

The successful execution of this task was verified by:
Checking the generated .txt label files.
Confirming normalized coordinate values between 0 and 1.
Ensuring correct grouping of annotations per image.
Observing successful completion without major errors.

Conclusion

The annotation data was successfully converted from CSV format to YOLO format. Each image now has a corresponding label file containing normalized bounding box coordinates, making the dataset ready for object detection model training in subsequent milestones.

CODE:

```
# Group by image_id
grouped = df_filtered.groupby("image_id")

for image_id, group in tqdm(grouped):

    label_file_path = os.path.join(LABEL_SAVE_PATH, image_id + ".txt")

    # Image size after resizing
    img_w, img_h = 1024, 1024

    with open(label_file_path, "w") as f:

        for _, row in group.iterrows():

            class_id = int(row["class_id"])

            x_min = row["x_min"]
            y_min = row["y_min"]
            x_max = row["x_max"]
            y_max = row["y_max"]

            # Convert to YOLO format
            x_center = ((x_min + x_max) / 2) / img_w
            y_center = ((y_min + y_max) / 2) / img_h
            width = (x_max - x_min) / img_w
            height = (y_max - y_min) / img_h

            f.write(f"{class_id} {x_center} {y_center} {width} {height}\n")

print("CSV → YOLO conversion completed.")
```

TASK NO. 4

Set up the development environment with PyTorch/TensorFlow, pydicom, pandas, OpenCV

Libraries used:

```
import os
import numpy as np
import pandas as pd
import pydicom
import cv2
from tqdm import tqdm
```

Environment Validation:

The development environment was validated by successfully:

- Importing all required libraries without errors.
- Loading the annotation CSV file (67,914 rows confirmed).
- Reading DICOM images using pydicom.
- Converting images to JPG format.
- Resizing images uniformly.
- Generating YOLO annotation files.
- Executing batch processing without runtime failures.

The successful execution of these operations confirms that the development environment was properly configured and fully functional.